

3

2026

Lycoris Radiata

BAEZA AMORÓS, ESTEFANÍA

[MEMORIA DEL PROYECTO FINAL DE CICLO: FLORISTERÍA ONLINE]

Memoria técnica sobre el desarrollo de una aplicación web para la gestión de una floristería con PHP y Tailwind CSS. Detalla las funciones para clientes (catálogo y pedidos personalizados) y administradores (panel de gestión CRUD), priorizando una interfaz sencilla, intuitiva y responsive.

Indice

Contenido

Indice	2
1.Introducción	3
2.Antecedentes.....	3
3. Análisis	4
4. Diseño y Arquitectura de la Aplicación	6
5. Resultados.....	16
6. Conclusiones	17

1.Introducción

El presente proyecto aborda el desarrollo de Lycoris Radiata, una aplicación web orientada a la gestión de pedidos y administración de una floristería online.

Destaca por haber sido diseñada y programada desde cero por la promotora, estudiante del ciclo de Desarrollo de Aplicaciones Web (DAW), lo que otorga a la empresa total autonomía tecnológica sin depender de proveedores externos.

Presentación del problema y objetivos: El crecimiento del comercio electrónico ha provocado que muchas empresas ofrezcan webs sobrecargadas y procesos de compra complejos que excluyen a muchos usuarios, especialmente a las personas mayores, aumentando la "brecha digital". El objetivo de este proyecto es solucionar este problema creando una plataforma minimalista, intuitiva y accesible. El sistema permitirá de forma sencilla: el registro de usuarios, la consulta del catálogo, la realización de pedidos personalizados y la administración total del negocio mediante operaciones CRUD.

Bondades de la solución propuesta: Además de su accesibilidad, la plataforma destaca por su compromiso con la sostenibilidad. Apuesta por un modelo colaborativo con invernaderos locales que garantiza la frescura del producto, apoya la economía local y reduce la huella de carbono derivada del transporte internacional.

2.Antecedentes

Realizando una investigación previa sobre varias páginas relacionadas con la venta de flores y la gestión de pedidos online (tales como [Interflora](#), [Aquarelle](#) y [1-800-Flowers](#)), se observaron distintas características comunes en la forma de presentar este tipo de plataformas.

Tras el análisis realizado, se constató que muchas de estas páginas presentan interfaces con una gran cantidad de información, categorías y elementos visuales, lo que puede hacer que la navegación resulte poco intuitiva y frustrante para usuarios con menor experiencia tecnológica.

Además, el contenido ofrecido por estas plataformas se basa principalmente en catálogos cerrados de ramos ya preparados, limitando las posibilidades de elección. Frente a esto, la propuesta de valor única de *Lycoris Radiata* es ofrecer una personalización 100% real "flor a flor". A través de un sistema guiado paso a paso, el cliente tiene la libertad creativa de elegir individualmente las flores exactas que conformarán su ramo (con precios accesibles por tallo), aportando un

mayor valor emocional al regalo y diferenciándose claramente de los e-commerce tradicionales.

3. Análisis

En este apartado se describen los principales elementos que componen la aplicación, los usuarios que interactúan con ella y el flujo general de funcionamiento del sistema. El objetivo es definir el comportamiento esperado de la plataforma antes de abordar aspectos relacionados con su implementación.

3.1 Descripción general del sistema La aplicación desarrollada es una plataforma web para la gestión de pedidos y administración de una floristería online. Su objetivo es centralizar la información del negocio, facilitando tanto la compra por parte de los clientes como la gestión interna de los productos y pedidos. El sistema está diseñado para ofrecer una experiencia sencilla e intuitiva, permitiendo a los usuarios consultar el catálogo disponible y realizar pedidos personalizados, mientras que los administradores disponen de herramientas para gestionar el contenido de la plataforma y supervisar los encargos realizados.

3.2 Roles de usuario La aplicación diferencia dos tipos principales de usuarios, cada uno con permisos y funcionalidades específicas:

- **Cliente:** El cliente puede registrarse, iniciar sesión y acceder al catálogo de productos. Desde ahí puede consultar flores y composiciones, realizar pedidos y revisar el estado de estos desde su cuenta.
- **Administrador:** El administrador gestiona el catálogo de productos mediante operaciones de alta, modificación y eliminación. También controla los pedidos realizados por los clientes y actualiza su estado según el proceso de preparación y entrega.

3.3 Flujo de la información El funcionamiento de la aplicación se basa en la interacción entre clientes y administradores a través de una única plataforma. En primer lugar, el administrador introduce y mantiene actualizados los productos disponibles en el catálogo. Estos productos pasan a estar visibles para los clientes, quienes pueden consultarlos y realizar pedidos a través de la aplicación. Una vez confirmado un pedido, la información queda registrada en la base de datos y pasa a estar disponible para su gestión desde el panel de administración. A partir de ese momento, el administrador puede actualizar su estado y realizar el seguimiento correspondiente. El cliente podrá consultar en cualquier momento la información asociada a sus pedidos desde su perfil de usuario.

3.4 Procesos lógicos Para el correcto funcionamiento de la aplicación se han identificado los siguientes procesos principales:

- **Gestión de pedidos:** registro, almacenamiento y seguimiento de los pedidos asociados a cada usuario.
- **Gestión de catálogo:** mantenimiento de los productos disponibles por parte del administrador.
- **Consulta de productos:** visualización de flores y composiciones disponibles para los clientes.

3.5 Requisitos del Sistema

Requisitos Funcionales (RF) Son las funciones específicas que el sistema debe ser capaz de realizar:

- **RF01 - Gestión de Usuarios y Roles:** El sistema debe permitir el registro y autenticación de usuarios, distinguiendo entre roles de 'Cliente' y 'Administrador'.
- **RF02 - Catálogo "Flor a Flor":** El sistema debe mostrar un catálogo dinámico donde los usuarios puedan buscar y seleccionar flores de forma individual.
- **RF03 - Gestión de Carrito y Pedidos:** El sistema debe permitir añadir productos a un carrito, tramitar la compra y consultar el historial de pedidos.
- **RF04 - Congelación de Precios:** Al tramitar un pedido, el sistema debe registrar el precio exacto de cada flor en ese momento específico en el histórico contable.
- **RF05 - Máquina de Estados de Pedidos:** El sistema debe gestionar el ciclo de vida de cada pedido mediante estados definidos (Pendiente, Enviado, Entregado, Cancelado).
- **RF06 - Panel de Administración (CRUD):** El administrador debe tener una interfaz dedicada para crear, leer, actualizar y eliminar flores, y modificar el estado de los pedidos.

Requisitos No Funcionales (RNF) Son las restricciones y atributos de calidad que el sistema debe cumplir:

- **RNF01 - Usabilidad y Accesibilidad:** La interfaz debe ser minimalista y libre de sobrecarga cognitiva.

- **RNF02 - Responsividad:** La aplicación debe adaptarse de forma óptima a cualquier tamaño de pantalla.
- **RNF03 - Seguridad:** Las rutas de gestión y modificación de pedidos deben estar protegidas mediante permisos de rol.
- **RNF04 - Eficiencia e Interactividad:** Las operaciones críticas (como eliminar elementos del catálogo) deben realizarse de forma fluida y asíncrona.

4. Diseño y Arquitectura de la Aplicación

4.1 Arquitectura de la Aplicación y Metodología de Desarrollo

Para el desarrollo de *Lycoris Radiata* se ha optado por una arquitectura de software basada en el patrón de diseño **MVC (Modelo-Vista-Controlador)**. Esta elección, implementada a través del framework Symfony (PHP), resulta fundamental para garantizar el principio de Separación de Responsabilidades (SoC - *Separation of Concerns*), permitiendo que el código sea modular, escalable y fácil de mantener.

La aplicación se estructura lógicamente en tres capas bien diferenciadas, cuya disposición se refleja directamente en el árbol de directorios del proyecto (/src y /templates):

- **El Modelo (Capa de Datos y Reglas de Negocio):** Es la capa encargada de interactuar con la base de datos y representar las entidades del negocio. En el ecosistema de Symfony, esta capa se divide en dos directorios clave:
 - src/Entity/: Contiene las clases puras de PHP que representan las tablas de la base de datos (tales como Flower.php, User.php, Order.php y OrderLine.php).
 - src/Repository/: Actúa como una capa de abstracción. Utilizando el ORM *Doctrine*, los repositorios (como FlowerRepository.php) centralizan todas las consultas a la base de datos, haciendo uso de herramientas como *QueryBuilder* para generar consultas SQL seguras y dinámicas de forma eficiente, aislando completamente el acceso a datos del resto de la aplicación.
- **La Vista (Capa de Presentación):** Es la interfaz con la que interactúa directamente el usuario (cliente o administrador). Toda la capa visual se gestiona mediante el motor de plantillas *Twig* y se localiza en el directorio /templates. Para mantener un diseño coherente, se ha creado una plantilla

maestra (base.html.twig) de la que heredan el resto de vistas organizadas por módulos (por ejemplo, catalog/index.html.twig para mostrar las flores, o admin/index.html.twig para el panel de gestión). El diseño visual de estas vistas se ha maquetado utilizando el framework de utilidades *Tailwind CSS*, garantizando un comportamiento totalmente adaptativo (*responsive*).

- **El Controlador (Capa de Lógica de Aplicación):** Actúa como intermediario y coordinador entre la Vista y el Modelo. Ubicados en el directorio src/Controller/, los controladores (como CatalogController.php o CartController.php) se encargan de interceptar las peticiones HTTP que realiza el usuario, procesar la información solicitando los datos necesarios a los Repositorios o a los Servicios externos, y finalmente retornar la respuesta renderizando la Vista correspondiente de Twig.

Esta estructuración en tres capas asegura que, por ejemplo, si en un futuro se desea cambiar la interfaz gráfica por completo, la lógica de la base de datos y de los controladores permanecerá intacta.

4.2 Estructura de Datos y Modelo Relacional

La persistencia de la información de *Lycoris Radiata* se gestiona mediante un Sistema de Gestión de Bases de Datos Relacional (RDBMS), utilizando el motor *InnoDB* para garantizar la integridad referencial y el soporte de transacciones, con una codificación de caracteres utf8mb4 para asegurar la correcta inserción de todo tipo de caracteres.

Aunque la interacción con la base de datos desde la aplicación se realiza a nivel de objetos mediante el ORM *Doctrine*, el esquema relacional subyacente se ha diseñado priorizando la normalización y la coherencia contable del comercio electrónico. La base de datos está compuesta por cuatro entidades principales:

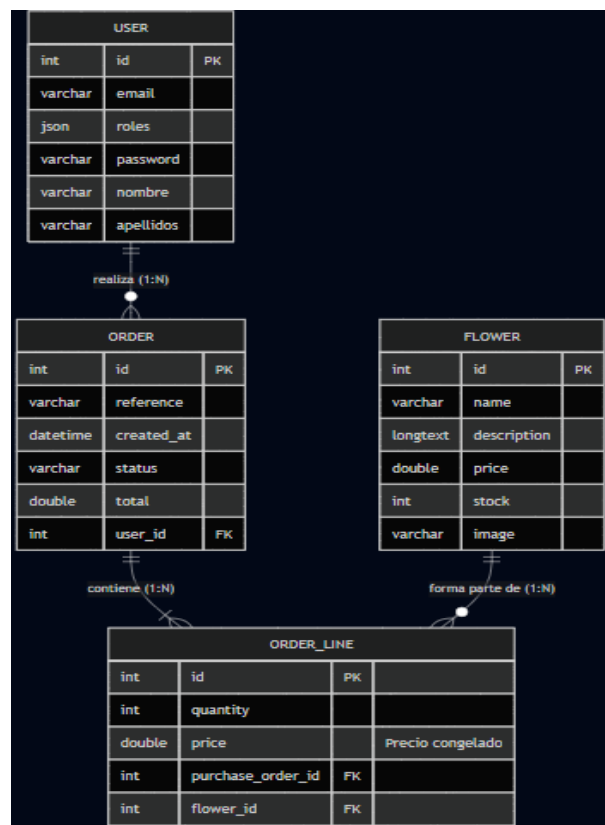
- **User (Usuario):** Entidad encargada de gestionar la autenticación y los datos de los clientes y administradores. Almacena de forma segura la contraseña encriptada (mediante algoritmos de *hashing* nativos de Symfony) y gestiona los permisos mediante el atributo roles, el cual almacena un *array* en formato JSON (ej. ["ROLE_ADMIN"]).
- **Flower (Flor):** Representa el inventario físico de la tienda. Contiene los datos descriptivos de cada producto, su imagen asociada y los valores dinámicos del negocio: el precio de venta (price) y el inventario disponible (stock).
- **Order (Pedido):** Actúa como cabecera o registro maestro de las transacciones. Está vinculada a un usuario mediante una relación de uno a muchos (1:N). Almacena la referencia única del pedido, la fecha exacta de

creación (created_at), el importe total y el estado actual dentro de la máquina de estados del ciclo de venta (ej. *PENDIENTE*, *ENTREGADO*).

- **Order_Line (Línea de Pedido):** Constituye el detalle desglosado del carrito de compras. Actúa como tabla intermedia que relaciona un pedido (purchase_order_id) con las flores adquiridas (flower_id).

Integridad Histórica de Precios: Una de las decisiones de diseño más críticas en la base de datos se encuentra en la tabla Order_line. Esta tabla posee su propio atributo price. Cuando un cliente finaliza una compra, la lógica de negocio consulta el precio actual en la tabla Flower y lo copia de forma inmutable en Order_line. Esto es un requisito fundamental en cualquier *e-commerce* real: garantiza que el histórico contable y de facturación de los pedidos anteriores no se corrompa si el administrador decide modificar el precio de las flores en el futuro.

A continuación, se muestra el Diagrama Entidad-Relación (ER) que ilustra la estructura y cardinalidad de la base de datos de la plataforma:



4.3 Módulos Lógicos y Servicios

La aplicación se ha modularizado con el objetivo de evitar la sobrecarga de los controladores (*fat controllers*) y favorecer la reutilización, la legibilidad y el mantenimiento del código. A nivel funcional, el sistema se divide en los siguientes módulos clave:

Módulo de Carrito de Compras (CartService) Uno de los núcleos técnicos del sistema es la gestión de las compras temporales. En lugar de procesar la lógica directamente en el CartController, se ha aplicado el principio de Separación de Responsabilidades creando un servicio dedicado (CartService.php).

Mediante la inyección de dependencias en su constructor, este servicio recibe la interfaz RequestStack (para acceder de forma segura a la sesión del usuario) y el FlowerRepository (para consultar el inventario). Para poder renderizar la vista de resumen o calcular los importes, el servicio reconstruye los objetos de la base de datos cruzando los identificadores guardados en sesión con el repositorio de Doctrine:

// Fragmento de código: Reconstrucción de entidades desde la sesión

```
public function getFullCart(): array
{
    $session = $this->requestStack->getSession();
    $carrito = $session->get('carrito', []);

    $carritoCompleto = [];

    foreach ($carrito as $id => $cantidad) {
        $flor = $this->flowerRepository->find($id);

        if ($flor) {
            $carritoCompleto[] = [
                'flor' => $flor,
                'cantidad' => $cantidad
            ];
        }
    }

    return $carritoCompleto;
}
```

Gracias a este enfoque, operaciones como el cálculo del total se realizan iterando sobre los objetos instanciados, manteniendo los controladores totalmente limpios.

Módulo de Seguridad y Administración El acceso a la gestión interna de la floristería está estrictamente protegido. En lugar de realizar comprobaciones manuales en cada función, se delega la seguridad al componente nativo de *Symfony* mediante el uso de atributos de PHP. Aplicando `#[IsGranted('ROLE_ADMIN')]` a la clase `AdminController`, se garantiza que cualquier intento de acceso sin privilegios sea bloqueado y redirigido automáticamente a nivel de enrutamiento.

Dentro de este módulo destacan funcionalidades como la subida segura de archivos (generando nombres únicos mediante la función `uniqid()`) y la eliminación asíncrona de flores del catálogo. Para mejorar la interactividad de la interfaz y evitar recargas completas de página, el controlador procesa el borrado en la base de datos, elimina la imagen física asociada del servidor mediante la función `unlink()`, y devuelve una respuesta estructurada en JSON que el *frontend* interpreta para actualizar el DOM dinámicamente:

// Fragmento de código: Borrado asíncrono en `AdminController`

```
#[Route('/admin/flor/borrar/{id}', name: 'app_admin_flower_delete', methods: ['DELETE'])]
public function delete(int $id, \App\Repository\FlowerRepository $flowerRepository, EntityManagerInterface $entityManager)
{
    $flor = $flowerRepository->find($id);

    if ($flor) {
        $nombreImagen = $flor->getImage();
        if ($nombreImagen) {
            $rutaImagen = $this->getParameter('kernel.project_dir').'/public/uploads/flower/'.$nombreImagen;
            if (file_exists($rutaImagen)) {
                unlink($rutaImagen);
            }
        }

        $entityManager->remove($flor);
        $entityManager->flush();

        return $this->json(['success' => true]);
    }

    return $this->json(['success' => false], 404);
}
```

4.4 Diseño de la Interfaz de Usuario

El diseño de la interfaz de *Lycoris Radiata* se ha fundamentado en el principio de usabilidad, buscando activamente mitigar la "brecha digital". Dado que el comercio electrónico tradicional suele presentar interfaces sobrecargadas, se ha optado por aplicar un minimalismo intencionado. La interfaz actúa como un lienzo en blanco (con toques en color pastel `#FF6F94` y tipografía *Great Vibes*) que

reduce drásticamente la carga cognitiva del usuario, cediendo el protagonismo visual a las flores sin perder la elegancia del tema.

A continuación, se detalla la estructura y el comportamiento de las vistas principales de la aplicación (desarrolladas mediante *Twig* y maquetadas con *Tailwind CSS*):

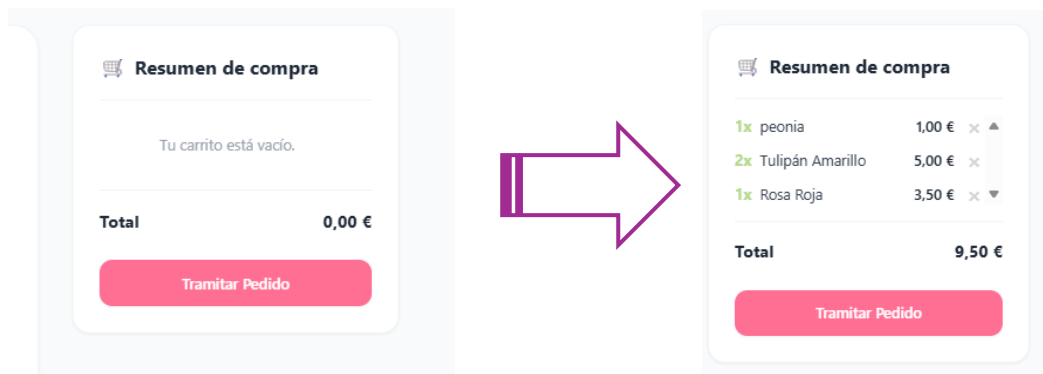
4.4.1 Página Principal (Landing Page) y Barra de Navegación El punto de entrada a la aplicación (home/index.html.twig) se ha diseñado para ser extremadamente directo. Consta de una página de inicio sencilla (*landing page*) cuyo objetivo principal es redirigir al usuario hacia la tienda mediante un botón de llamada a la acción ("Ir al catálogo").



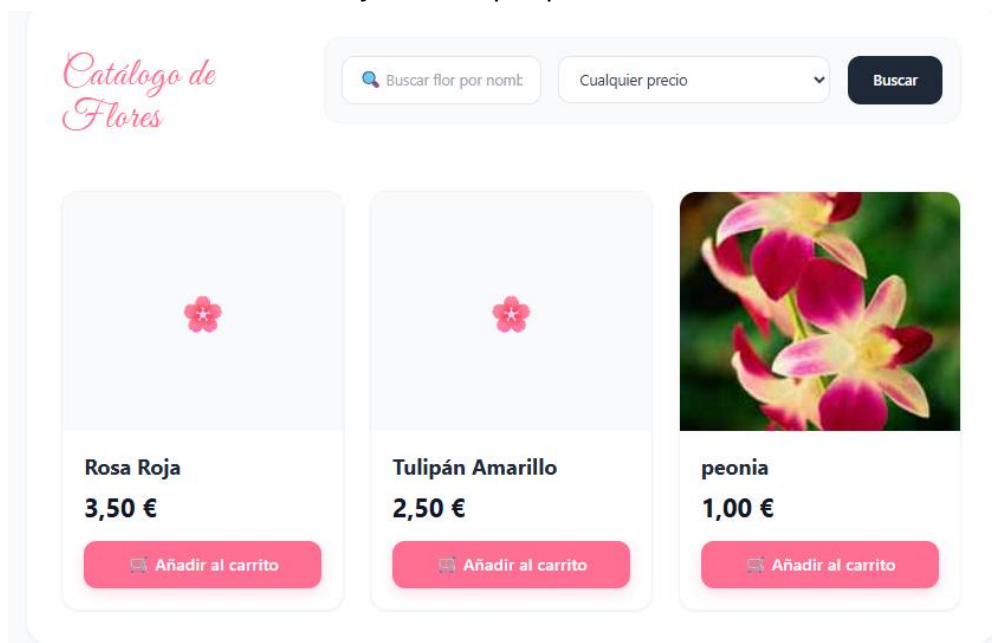
En la parte superior, se ha implementado una barra de navegación (*Navbar*) común a todas las vistas. Esta barra contiene el título de la página, un enlace directo al catálogo y un menú de perfil desplegable para gestionar la sesión del usuario. Este es visible en toda la aplicación



4.4.2 Catálogo Dinámico y Retroalimentación Visual La vista principal de compra (catalog/index.html.twig) muestra el inventario de flores disponibles. Para mejorar la experiencia de usuario (UX) durante el proceso de compra "flor a flor", se ha implementado un sistema de retroalimentación inmediata. En la versión de escritorio, junto a la cuadrícula del catálogo, se visualiza un resumen en forma de lista que se actualiza mostrando los elementos que el usuario va añadiendo.



La zona del catalogo dispone de las tarjetas de las flores. Estas muestran el nombre y el precio de la flor junto con el botón de añadir flor que interactúa directamente con el resumen del carrito. También muestra dos filtros sencillos: un buscador de nombres y un filtro por precio.



Esta misma pantalla, si es un administrador quien está logueado, muestra también un botón para eliminar flores y para añadir nuevas y la cantidad de stock.

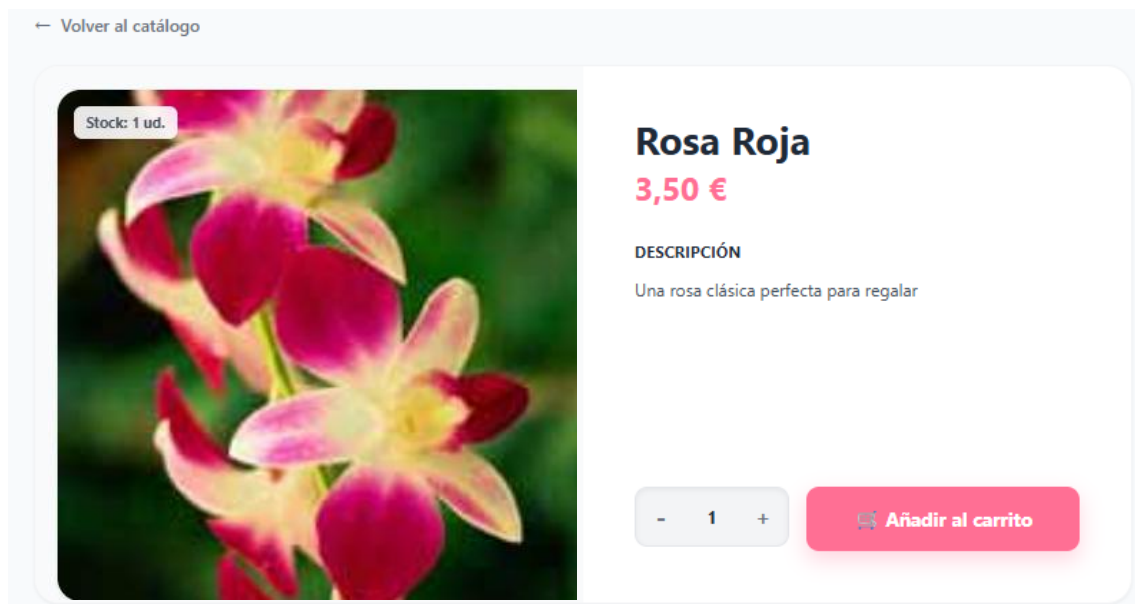


Este lleva a un formulario sencillo donde se rellenan los datos de la nueva flor



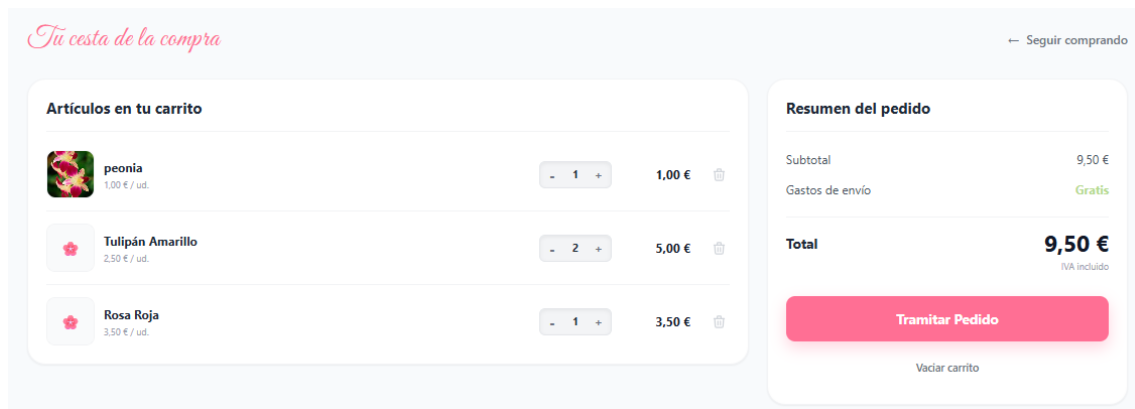
Adaptación a Dispositivos Móviles (Diseño Responsive): Gracias a las clases utilitarias de *Tailwind* CSS, la interfaz se adapta fluidamente a pantallas pequeñas. Para optimizar el espacio en dispositivos móviles, la lista resumen lateral del catálogo se oculta automáticamente y es reemplazada por una burbuja flotante fijada en la parte inferior de la pantalla, la cual indica de forma visual e intuitiva la cantidad total de elementos añadidos a la lista.

4.4.3 Detalle del Producto y Carrito de Compras Para aquellos usuarios que requieran más información antes de la compra, se ha diseñado una vista de detalle (catalog/detail.html.twig). Esta además de lo anteriormente mencionado muestra también la descripción de la flor



Cabe mencionar que si la flor no tuviera imagen, no sería problema, ya que muestra un emote de flor en su defecto

Por otro lado, la vista del carrito (cart/index.html.twig) consolida la selección del usuario antes de la tramitación final, mostrando un resumen claro de las cantidades y el importe total calculado por el CartService.



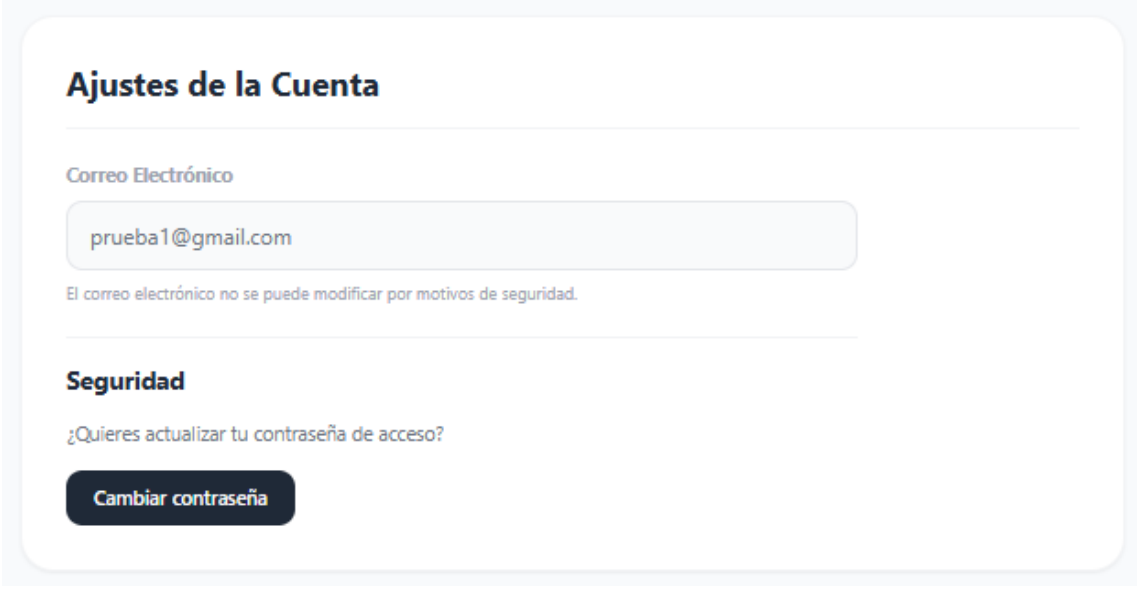
4.4.5 Perfil: Mis pedidos y Configuración.

El área privada del cliente se ha diseñado para ofrecer transparencia y control sobre su cuenta, manteniendo la misma estética limpia y libre de sobrecarga cognitiva del resto de la plataforma. Esta sección se organiza en dos áreas principales: el historial de compras y los ajustes de la cuenta.

En el apartado de **Mis pedidos**, el usuario puede consultar un registro detallado de todas sus transacciones. Para facilitar la rápida comprensión visual de la información, el progreso del pedido se refleja utilizando la máquina de estados interna de la aplicación (ej. *Pendiente*, *Enviado*, *Entregado*). Estos estados se representan mediante identificadores visuales claros o etiquetas de colores (*badges*) maquetadas con *Tailwind CSS*, permitiendo al cliente conocer la situación exacta de su ramo de un solo vistazo. Además, el historial desglosa el coste exacto de cada flor en el momento de la compra, demostrando en la vista la integridad contable y la congelación de precios implementada en la base de datos.



Por su parte, la vista de **Configuración** (profile/settings.html.twig) aloja los formularios para la modificación de los datos personales y credenciales de acceso. Siguiendo la filosofía de accesibilidad para evitar la exclusión digital, este formulario se ha simplificado al máximo, mostrando campos de texto de gran tamaño y botones de acción muy descriptivos, evitando menús complejos o pestañas ocultas que puedan confundir al usuario.



Ajustes de la Cuenta

Correo Electrónico

prueba1@gmail.com

El correo electrónico no se puede modificar por motivos de seguridad.

Seguridad

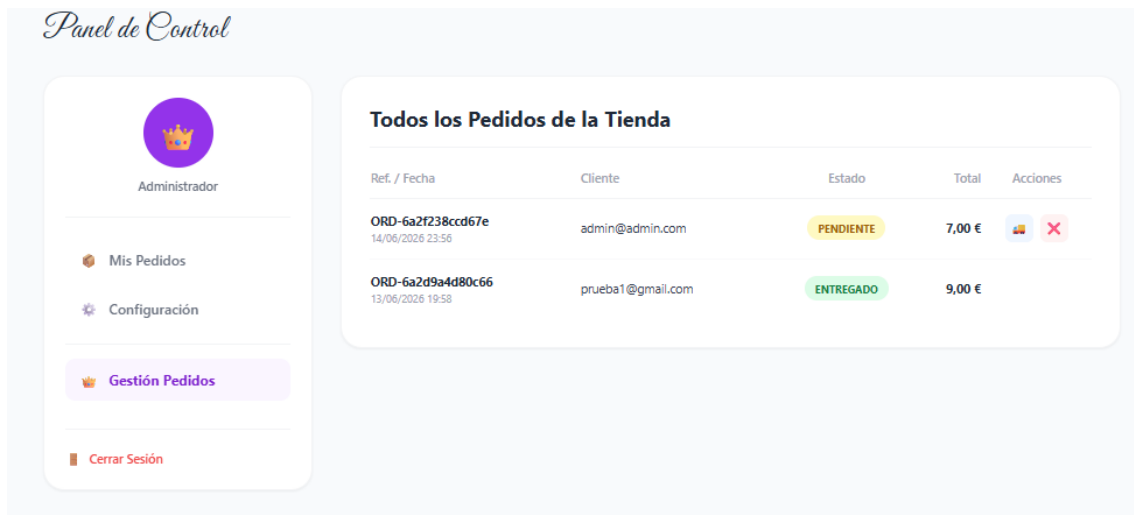
¿Quieres actualizar tu contraseña de acceso?

Cambiar contraseña

4.4.6 Panel de Administración (Backend)

La interfaz de gestión (admin/index.html.twig), restringida exclusivamente a la promotora, abandona el enfoque visual y comercial de la tienda para centrarse en la productividad y el flujo de trabajo. El núcleo central de este panel es la **gestión integral de los pedidos** realizados por los clientes.

El elemento más destacado de esta pantalla es el control de la máquina de estados. Para optimizar el tiempo de la promotora, la interfaz incorpora controles ágiles que le permiten actualizar la situación logística de cada pedido (alternando entre *Pendiente*, *Enviado* o *Entregado*). Estos cambios se reflejan instantáneamente de forma muy visual mediante el uso de etiquetas de colores o *badges* diseñados con *Tailwind CSS*, permitiendo a la administradora conocer el volumen de trabajo pendiente de un solo vistazo y sin recargar la pantalla con información innecesaria.



5. Resultados

En esta fase se ha evaluado el sistema para comprobar el grado de cumplimiento de los objetivos descritos en el primer capítulo. Tras las pruebas realizadas, se puede afirmar que los objetivos principales se han alcanzado satisfactoriamente:

- **Autonomía y Gestión Centralizada:** Se ha logrado desarrollar una plataforma web totalmente funcional desde cero, permitiendo el registro de usuarios, la consulta de productos y la administración íntegra del catálogo mediante un panel protegido.
- **Accesibilidad y Usabilidad:** El diseño minimalista implementado con *Tailwind CSS* ha dado como resultado una interfaz limpia, responsiva y libre de sobrecarga cognitiva, cumpliendo el objetivo de mitigar la brecha digital para usuarios de avanzada edad.
- **Personalización:** El servicio de carrito de compras funciona de manera óptima, permitiendo al cliente la selección individual de productos y garantizando la integridad de los precios (congelación de importes) al formalizar el pedido.

Durante el desarrollo, se tomó la decisión técnica de aplazar la implementación de la pasarela de pago real con tarjeta y el sistema de notificaciones automáticas por correo electrónico. Siguiendo las directrices de desarrollo de software para este tipo de proyectos, se consideró preferible asegurar la entrega de módulos base completos y libres de errores (como el carrito y el CRUD de administración) en lugar de presentar múltiples módulos incompletos. Estas funcionalidades pendientes han quedado debidamente documentadas para su futura implementación.

6. Conclusiones

6.1 Descripción del desarrollo del proyecto El desarrollo de *Lycoris Radiata* se ha llevado a cabo dentro del tiempo estimado para este tipo de proyectos, dedicando aproximadamente unas 90 horas de trabajo efectivo.

A lo largo del ciclo de vida del software, los principales problemas técnicos encontrados estuvieron relacionados con la curva de aprendizaje del framework *Symfony* y la gestión de la base de datos. Específicamente, requirió un gran esfuerzo aislar la lógica del carrito de compras utilizando sesiones asíncronas para no sobrecargar los controladores, así como diseñar las consultas dinámicas a través de *Doctrine ORM* para garantizar que el borrado de elementos en la base de datos se reflejara de manera instantánea (vía AJAX/Fetch) en el panel del administrador sin generar inconsistencias.

6.2 Conclusiones del trabajo realizado El resultado final es una aplicación web robusta, escalable y orientada a resolver un problema real del comercio electrónico moderno. La adopción del patrón de arquitectura Modelo-Vista-Controlador (MVC) ha sido un acierto, ya que ha permitido mantener un código limpio, estructurado y fácil de mantener. A nivel personal y profesional, el proyecto ha servido para asentar conocimientos avanzados en PHP orientado a objetos, el manejo de bases de datos relacionales y la maquetación moderna con *Tailwind CSS*, cumpliendo sobradamente con las expectativas y exigencias del ciclo formativo.

6.3 Trabajos futuros Dado que la aplicación cuenta con una base sólida y escalable, se proponen las siguientes ampliaciones como trabajos futuros:

- **Pasarela de Pagos:** Integración de una API de pagos de terceros (como *Stripe* o *PayPal*) para procesar transacciones reales de manera segura.
- **Notificaciones Automáticas:** Implementación del componente *Mailer* de *Symfony* para enviar correos electrónicos de confirmación al cliente cada vez que el administrador cambie el estado de su pedido.
- **Generación de Informes PDF:** Creación de un módulo de analítica web para el administrador que permita exportar balances de ventas e inventario en formato PDF.
- **Autenticación MFA (Multifactor):** Implementación de una capa adicional de seguridad en el inicio de sesión, requiriendo un segundo paso de verificación (vía correo electrónico o *Google Authenticator*). Esto resulta especialmente crítico para blindar el acceso con el rol de Administrador y proteger los datos de ventas y clientes de la floristería.